

Árboles de Clasificación y Regresión (CART)

Tutorial 11 – Recursive binary splitting, pruning y aplicaciones en economía

Luciana Azul Ramirez

E337 – Big Data para Economistas
Universidad de San Andrés

Otoño 2026

Hoja de ruta de la clase

Primera parte: la herramienta del día

- 1 **¿Qué problema viene a resolver CART?** Conexión con OLS y con los métodos lineales que ya vimos.
- 2 **Árboles de regresión y de clasificación:** terminología, algoritmo, criterios de impureza.
- 3 **Pruning:** cost-complexity y cómo elegirlo por cross-validation.
- 4 **CART vs. modelos lineales:** cuándo gana cada uno, ventajas y desventajas.

Segunda parte: papers aplicados

- 1 **Varian (2014, JEP)** – “Big Data: New Tricks for Econometrics”. CART aplicado al Titanic y a datos de hipotecas (HMDA).
- 2 **Keely & Tan (2008, J. Public Econ.)** – CART para identificar grupos de identidad con preferencias distintas por redistribución del ingreso.

Tercera parte: notebook en Python

Tut11_CART.ipynb con `scikit-learn`: `DecisionTreeClassifier` y `DecisionTreeRegressor`.

Parte I

CART: idea, algoritmo, pruning y comparación con métodos lineales

¿Qué problema viene a resolver CART?

- Hasta ahora vimos sobre todo métodos *lineales*: OLS, LASSO, Ridge, PCR, PLS.
- Esa caja de herramientas **se queda corta** cuando:
 - el efecto de una variable depende *fuertemente* del valor de otra (**interacciones**);
 - la relación entre Y y los X es **no lineal** y cambia de forma según la región del soporte.
- **CART** (Breiman et al. 1984) hace algo radicalmente distinto: en vez de buscar una fórmula global tipo $X'\beta$, **parte el espacio** de los predictores en pedazos y dentro de cada pedazo predice algo *simple* (un promedio o una clase modal).

Las tres preguntas guía

- (1) ¿Qué problema? Capturar no linealidades e interacciones *sin tener que especificarlas a mano*.
- (2) ¿Cuál es la idea? Partir el soporte en regiones y predecir constante en cada una.
- (3) ¿Cuándo sí y cuándo no? Lo respondemos al final de la Parte I.

- **Nodo raíz:** la muestra completa, antes de partir nada.
- **Nodo interno:** punto donde el árbol se divide preguntando " $X_j \leq s$ ".
- **Hoja** (*nodo terminal*): donde el árbol ya no se divide más; ahí se hace la predicción.
- **Profundidad:** cuántas preguntas como mucho hay que responder hasta llegar a una hoja.

Idea clave

Cada hoja es un **cuadrante** del espacio de predictores. Dentro de cada cuadrante el árbol predice *lo mismo* para todas las observaciones:

- **regresión:** el promedio de Y en la hoja;
- **clasificación:** la clase modal de la hoja.

¿Cómo elige el árbol dónde partir?

La pregunta: con Y continua, ¿qué corte $X_j \leq s$ tomamos?

Recursive binary splitting (*greedy*)

En cada nodo, el algoritmo prueba **todas** las variables j y **todos** los cortes posibles s , y elige el par que deja *las dos hojas hijas con menor varianza interna* (suma de cuadrados respecto del promedio de cada hoja).

- **Greedy:** nunca mira hacia adelante. Elige el mejor corte *ahora*, sin preguntarse si más adelante habría una partición mejor.
- **Por qué greedy:** encontrar la partición globalmente óptima es combinatorio $\Rightarrow$ inviable. Greedy es una aproximación que funciona bien en la práctica.
- Una vez que el nodo se parte, el algoritmo **repite el mismo procedimiento** en cada hoja hija, hasta cumplir un criterio de *stopping*.

Y en clasificación, ¿qué minimizamos?

Con Y continua minimizábamos la varianza dentro de la hoja. Con Y categórica queremos algo análogo: una hoja *buena* es una hoja **pura** (todos los casos de la misma clase).

Impureza de Gini (la que usa `sklearn` por default)

En un nodo, si $\hat{p}_k$ es la proporción de casos de la clase k :

$$G = \sum_k \hat{p}_k (1 - \hat{p}_k).$$

- **Gini** = 0 $\Leftrightarrow$ nodo puro (todos de la misma clase).
- **Gini máximo** $\Leftrightarrow$ clases mezcladas en partes iguales.
- El árbol elige el corte que más reduce la Gini ponderada de las dos hojas hijas.
- Hay alternativas (*entropía* / *cross-entropy*, error de clasificación) que dan resultados **muy similares** en la práctica. Lo vamos a ver en el notebook.
- Predicción en una hoja: la **clase modal**.

Stopping y el problema del overfitting

Tres criterios típicos de *stopping* (controlan el tamaño del árbol):

- **Profundidad máxima** (`max_depth`): cuántos *pisos* puede tener el árbol.
- **Mínimo de observaciones por hoja** (`min_samples_leaf`): no se aceptan hojas con menos de $n_{\text{mín}}$ observaciones.
- **Mejora mínima** en la impureza para aceptar un *split*.

El problema

Sin restricciones, el árbol **crece hasta hojas con 1 observación**: *overfitting* masivo. Predice perfecto en *train* y muy mal fuera de la muestra (alta varianza).

- Solución *prematura* (pre-pruning): fijar `max_depth` o `min_samples_leaf` chico y elegirlos por CV.
- Solución *post-hoc* (post-pruning): **cost-complexity pruning** $\Rightarrow$ siguiente slide.

Cost-complexity pruning (*podar* el árbol)

Idea: dejar crecer un árbol grande y después *podarlo*, eligiendo el tamaño que mejor generaliza.

La analogía con LASSO

Penalizamos al árbol por tener muchas hojas:

$$\underbrace{\text{ajuste in-sample}}_{\text{SSR / impureza}} + \alpha \cdot (\text{nro. de hojas}).$$

α acá juega *exactamente el mismo rol* que λ en LASSO o Ridge: es un **hiperparámetro de regularización**.

- $\alpha = 0$: árbol gigante (overfit, baja sesgo, alta varianza).
- α grande: árbol degenerado a la raíz (predice el promedio global; alto sesgo, baja varianza).
- Se elige α por **k-fold cross-validation**, igual que el λ de LASSO/Ridge.
- En `sklearn` este hiperparámetro se llama `ccp_alpha`.

Hiperparámetros que veremos en sklearn

Argumento	Qué controla	Comentario
<code>criterion</code>	Medida de impureza	'gini' (default) o 'entropy' para clasificación; 'squared_error' para regresión.
<code>max_depth</code>	Profundidad máxima	Pre-pruning. None = sin límite (overfitting garantizado).
<code>min_samples_leaf</code>	Tamaño mínimo de hoja	Pre-pruning. Útil con datasets ruidosos.
<code>min_samples_split</code>	Mínimo para dividir un nodo	Pre-pruning, similar al anterior.
<code>ccp_alpha</code>	Costo de complejidad α	Post-pruning. 0.0 = no pruning.

- Todos se eligen por **cross-validation**, típicamente con GridSearchCV de `sklearn.model_selection`.
- Buena práctica: *no* estandarizar X (a CART no le importa la escala: solo le importa el *orden*).
- No hay que crear dummies para categóricas binarias; sklearn sí necesita codificar las categóricas multinivel.

CART vs. modelos lineales

	Modelo lineal (OLS)	CART
Forma funcional	$E[Y X] = X'\beta$ (global)	Constante a trozos en regiones rectangulares
Interacciones	Hay que <i>especificarlas</i> a mano	Las captura <i>automáticamente</i>
No linealidades	Hay que transformar X	Las captura <i>automáticamente</i>
Categóricas	Dummies a mano	Maneja directamente (en R; en sklearn hay que codificar)
Interpretación	Coeficientes con efectos marginales	Reglas de decisión (<i>si $X_1 \leq s$, entonces ...</i>)
Inferencia	SE, t -tests, etc.	No estándar (sin teoría asintótica simple para los splits)

- Si $E[Y | X]$ es *realmente* lineal, OLS le gana a CART (menos varianza).
- Si la verdadera estructura tiene *escalones* o *interacciones fuertes*, CART le gana a OLS.
- En la práctica: probar ambos, comparar por validación cruzada.

Ventajas y desventajas

Ventajas

- **Interpretable:** el árbol se lee como un conjunto de reglas *if-then*.
- Captura **interacciones** y **no linealidades** sin imponer una forma funcional.
- Maneja categóricas y mezclas de tipos de datos.
- No requiere estandarizar.
- Manejo natural de **missing values** (en R; en `sklearn` requiere imputar).

Desventajas

- **Inestable:** cambios pequeños en los datos producen árboles *muy* distintos (alta varianza).
- Predicción *discontinua*: saltos en los bordes de las regiones.
- Performance predictiva **inferior** a métodos lineales cuando la verdad es lineal.
- Sin teoría inferencial estándar (no hay p -values usuales sobre los splits).
- Sesgo hacia variables *con muchos niveles* (más posibles cortes).

- La **inestabilidad** es la motivación de los *métodos de ensamble* (*bagging*, *random forests*, *boosting*) que veremos en el Tutorial 12.

Parte II

Dos papers de la bibliografía obligatoria

Primer Trabajo

Hal R. Varian (2014)

Big Data: New Tricks for Econometrics

Journal of Economic Perspectives, 28(2), 3–28.

- **Contexto:** Hal Varian (Chief Economist en Google) escribe en *JEP* un *survey* pedagógico para economistas.
- **Pregunta del paper:** ¿qué herramientas de *machine learning* debería conocer un economista aplicado, y para qué problemas sirve cada una?
- Recorre: **árboles (CART)**, *ctrees*, *random forests*, *bagging/boosting*, regularización (LASSO, elastic net), *Bayesian Structural Time Series*.
- **Por qué importa para nuestro tema:** es la **introducción canónica** a CART desde una óptica econométrica. Muestra dos ejemplos vivos con código en R.

Mensaje general del paper

Las técnicas de ML *complementan* la caja de herramientas econométrica clásica: son particularmente útiles para **predicción out-of-sample** y para **detectar no linealidades e interacciones** que un modelo lineal pasaría por alto.

Aplicación 1: Titanic — supervivientes

- **Datos:** pasajeros del Titanic. Y = sobrevivió (sí/no). Predictores en el ejemplo simple: *class* (categórica con 3 niveles) y *age*.
- **Por qué este caso:** hay una **interacción fuerte** entre clase y edad (“mujeres y niños primero, particularmente en 1.^a clase”) que OLS / logit *no* capturan bien.
- Árbol estimado con la librería *rpart* en R:
 - Primera división: $class \geq 2.5$ (3.^a clase vs. resto).
 - En 3.^a clase: predice *died* (370/501).
 - Entre 1.^a y 2.^a clase: división por $age \geq 16$; niños sobreviven (34/36).
 - Adultos: nueva división por clase; 2.^a clase $\rightarrow$ *died*, 1.^a clase $\rightarrow$ *lived*.

Comparación con logit

Una *logit* de *survived* sobre *age* estima un coeficiente **no significativo** (la edad “no importa”). Pero el árbol revela que la relación es **no monótona**: sobreviven niños y, en menor medida, ancianos $\Rightarrow$ *el logit oculta una no linealidad importante*.

- Munnell et al. (1996, AER): *¿hay discriminación racial en aprobación de hipotecas en Boston?* Usan un **logit** con ~ 12 predictores: estiman un coeficiente sobre raza *negativo y significativo*.
- Varian **re-estima** con un *ctree* (*conditional inference tree*) sobre los mismos datos.
- **Resultado del árbol:**
 - Variable más importante: *dmi* (*denied mortgage insurance*).
 - La variable raza aparece **muy al fondo** del árbol.
 - Misclassification del árbol: 9.6 % vs. 9.5 % del logit.
 - Si *sacamos* la variable raza, las predicciones del árbol **no cambian**.
- Esto **no** prueba ausencia de discriminación (puede operar en variables correlacionadas con raza), pero *sí* sugiere que el efecto marginal del logit no es robusto a especificaciones no lineales.

Tres lecciones para nosotros

- **CART y logit responden preguntas distintas:** el logit estima un *efecto marginal lineal en log-odds*; el árbol estima una *regla de predicción no lineal*.
 - En presencia de **no linealidades o interacciones**, un coeficiente lineal puede ser *engañoso*: “ausencia de efecto” en el logit no implica ausencia de relación.
 - La **comparación de modelos out-of-sample** (no R^2 in-sample) es la métrica correcta.
-
- Varian arranca con CART pero *anticipa* que los árboles individuales son *inestables*: hay que pasar a **ensambles** (bagging, random forests, boosting) para ganar capacidad predictiva.
 - Nota cultural: este paper es *la* cita estándar cuando un economista quiere justificar el uso de ML en un trabajo aplicado.

Segundo Trabajo

Louise C. Keely & Chih Ming Tan (2008)

Understanding Preferences for Income Redistribution

Journal of Public Economics, 92, 944–961.

Pregunta y motivación

- Hay dos familias de teorías sobre por qué la gente prefiere o no la redistribución del ingreso:
 - *Preference-based* (Alesina, La Ferrara): la **etnicidad** es el clivaje principal.
 - *Information-based* (Benabou–Ok, Piketty): la identidad funciona como *proxy* de información sobre la propia trayectoria de ingresos $\Rightarrow$ “Prospect of Upward Mobility”.
- Estas teorías predicen **distintos grupos de identidad** con preferencias distintas. ¿Cuál es consistente con los datos?
- **Problema metodológico**: la literatura previa elige *a priori* las variables relevantes (raza, género) y a lo sumo prueba **interacciones bivariadas**. *No* explora sistemáticamente todo el espacio de posibles particiones.

Pregunta empírica

¿Qué grupos de identidad — definidos por combinaciones de raza, edad, género, educación, religión, región — están sistemáticamente asociados con distintas preferencias por redistribución?

- **Fuente:** General Social Survey (GSS), pooled 1978–2000. $n \approx 13\,000$ encuestados.
- **Variable de identidad** (predictores X): age, sex, race, region of birth, born in US, parents born in US, mother's education, religion at 16, fundamentalism, year. Mezcla de continuas, binarias y categóricas multinivel.
- **Variables de preferencias** (outcome Y , categórica):
 - EQWLTH: escala 1–7. “¿El gobierno debería reducir las diferencias de ingreso?”
 - NATFARE: escala 1–3. “¿Gastamos demasiado/poco en welfare?”
 - Otras: INCGAP, GOODLIFE, PARSOL...
- **Problema clave:** el número de *posibles* subgrupos con todas las interacciones es enorme. Un modelo lineal con dummies $\times$ dummies es inviable.

¿Cómo usan CART (y random forests)?

- **Estrategia:** dejar que CART *descubra* qué combinaciones de variables de identidad son las que mejor predicen las preferencias por redistribución.
- Cada **hoja** del árbol = un *grupo de identidad* caracterizado por una conjunción de condiciones ($X_j \leq s_j$).
- Para cada hoja reportan: descripción de la conjunción (las A_j en sus tablas) y la **predicción modal** para esa hoja.
- Usan **Random Forests** en paralelo para dos cosas:
 - *Variable importance scores*: rankear qué variables de identidad son las más relevantes.
 - *Validar* las predicciones del árbol simple (menos varianza).
- Pruning vía *10-fold cross-validation* (cost-complexity, $V = 10$).

Por qué CART es la herramienta “natural” acá

El paper **no** quiere imponer *a priori* qué interacciones existen. CART las identifica automáticamente. Y el árbol es **interpretable**: las hojas son grupos demográficos concretos.

Resultado central: el árbol para EQWLTH

Variable	Grupo 1	Grupo 2	Grupo 6	Grupo 8
RACE	Black + others	White	White	White
MADEG (educ. madre)	All	Less HS	HS-graduate	HS-graduate
AGE	All	< 44	> 25	> 36
SEX	All	All	Male	Female
EQWLTH predicho	1 (pro-redistr.)	1	7 (anti)	6,7

- Variable *más importante*: **RACE** (score = 100). Le siguen **AGE** (82.9), **SEX** (69.2), **MADEG** (48.8).
- No blancos: bloque *único* y **fuertemente pro-redistribución**, sin más subdivisiones.
- Blancos: el árbol se subdivide en 7 hojas según edad, género y educación de la madre.
- Patrón: blancos jóvenes con madre poco educada → pro-redistribución; blancos mayores con madre educada → contra.

Hallazgos sustantivos

- Las teorías existentes (preference-based y information-based) **no** explican del todo los patrones.
- Los *groupings* relevantes son **más complejos** que “negros vs. blancos” o “ricos vs. pobres”.
- Aparecen *interacciones* entre raza, ciclo de vida y *background* socioeconómico que la literatura anterior no había documentado.

Mensaje metodológico para nosotros

- CART como herramienta *exploratoria*: cuando uno *no sabe* qué interacciones importan, dejar que el algoritmo las descubra.
- La **interpretabilidad** del árbol es clave en economía aplicada: cada hoja es un grupo demográfico concreto, no un “score” opaco.
- **Limitación honesta**: CART no da inferencia estándar $\Rightarrow$ los autores acompañan con *random forests* para validar robustez.

Resumen: dos papers, dos mensajes

Paper	Uso de CART	Mensaje
Varian (2014, JEP)	Pedagógico: Titanic y HMDA	CART revela no linealidades (edad-supervivencia) e interacciones (clase-edad) que un logit pasa por alto.
Keely & Tan (2008, JPE)	Sustantivo: GSS, preferencias por redistribución	CART identifica grupos demográficos con preferencias heterogéneas <i>sin imponer</i> a priori cuáles son las interacciones relevantes.

Patrón común

En **ambos** casos, CART aporta donde el modelo lineal falla: cuando hay *no linealidades fuertes* o *interacciones de alto orden* que el investigador no quiere imponer a mano. Y en ambos, los autores anticipan que para **ganar capacidad predictiva** hay que pasar a métodos de ensamble (Tutorial 12).

Transición al notebook

Ahora vamos a implementar CART en Python sobre dos datasets.

Notebook: Tut11_CART.ipynb

Datos:

- Titanic (de Kaggle, `train.csv/test.csv`): clasificación binaria *survived*.
- Hitters (ISLP): regresión sobre el salario de jugadores de la MLB.

Herramientas que vamos a usar:

- `sklearn.tree.DecisionTreeClassifier` y `DecisionTreeRegressor`.
- `sklearn.tree.plot_tree` para visualizar el árbol.
- `sklearn.model_selection.KFold` y `GridSearchCV` para elegir `max_depth`, `min_samples_leaf` y `ccp_alpha` por cross-validation.
- `sklearn.metrics.accuracy_score` (clasificación) y `MSE` (regresión).

- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An introduction to statistical learning: with applications in Python*. Cap. 8.1. Springer.
- Friedman, J., Hastie, T., & Tibshirani, R. (2001). *The elements of statistical learning*. Cap. 9.2. Springer.
- Keely, L. C., & Tan, C. M. (2008). Understanding preferences for income redistribution. *Journal of Public Economics*, 92(5–6), 944–961.
- Sosa Escudero, W. (2021). *Big Data*, 7.^a edición, Siglo XXI, Cap. 3, pp. 85–94.
- Varian, H. R. (2014). Big data: New tricks for econometrics. *Journal of Economic Perspectives*, 28(2), 3–28.